CS 6120 Assignment 4

Hoang Nguyen Bich Ngoc (Norah) - 002537880

October 26, 2026

Design choices

- Special tokens are added in `count_n_grams`, not during preprocessing, since the padding depends on n. Each sentence gets (n - 1) `<s>` tokens.
- Possible next words are the vocabulary plus `<e>` and `<unk>`. `<s>` is left out because it never follows a word.
- For ties in `predict_next_word`, the word that appears first in the training data wins.

Starter code changes

- `make_probability_matrix`: replaced `unique_words` with `vocabulary`, since `unique_words` isn't defined in the function.
- I followed the starter code's `NGramModel` rather than the assignment PDF's version. The PDF's version only stores `vocab_size`, but `estimate_probabilities` needs the actual word list to build its output dictionary.
- `estimate_probabilities`: updated the docstring's Args to match the actual parameters.

Extra credit: StyleGram

Constants

- `STYLE_N = 3`: two words of context are more likely to catch phrases typical of each author, like "thou art." Bigrams see too little, and larger n may make contexts too unique to have any counts.
- `COUNT_THRESHOLD = 3`: rare words often show style, so I keep most of them. Words seen fewer than 3 times across all three books are dropped as noise.
- $k = 1$: same as Q3, and it keeps every probability above zero, since $\log 0$ is undefined.
- `split_ratio = 0.8`: same split as Q1. The held-out 20% is used for evaluation.

Design

- All styles share one vocabulary, built from the three training sets together. With separate vocabularies, a Shakespeare passage was labeled Hemingway: words he never used became `<unk>`, which scored well, and a smaller $|V|$ gave unseen n-grams higher smoothed probabilities.
- Context counts are built by adding up the trigram counts for each context. With $n-1$ padding, `('<s>', '<s>')` was never counted, so the first word of every line was scored against 0.

Classifying the style

I score the passage with each author's model by adding up $\log P(w \mid \text{context})$, and the highest score wins. A softmax turns the scores into the style probability.

I chose this over logistic regression because:

- The models already exist for prediction, so no extra training.
- It uses word order, not just which words appear.
- The same model picks the style and predicts the words.

Tradeoff: score gaps are large, so the style probability is often near 1, even when the match is unclear.

Prediction

- The top k words come from the winning model, using the last $n-1$ tokens of the last line as context. Results are lists, sorted from most to least likely.
- If the last two words (context tokens for a trigram model) never appear together in the training text, every next word gets the same probability, so the top k words just follow vocabulary order instead of highest probability. Backing off to a bigram model with a unigram context might help.

Evaluation

I hold out 20% of each file and score 5-line passages from it. The classifier picks the right author for 95.1% of them in the 1st run.